Shiv Nadar University Chennai

Degree & Branch	B.Tech Artificial Intelligence & Data Science	Semester V
Subject Code & Name	CS3807 – Deep Learning Laboratory	AY: 2026–27

Experiment 3

Implementation of Convolutional Neural Networks (CNNs) for Image Classification

1 Objective

To understand the working principle of Convolutional Neural Networks by implementing convolution, pooling, feature map visualization, and image classification using TensorFlow/Keras.

2 Learning Outcomes

After completing this experiment, the student will be able to:

1. Explain the convolution operation.
2. Compute output dimensions using stride and padding.
3. Visualize feature maps.
4. Implement max pooling and average pooling.
5. Calculate CNN parameters.
6. Build a CNN for image classification.
7. Evaluate CNN performance.

3 Background Theory

A Convolutional Neural Network (CNN) is a deep learning architecture used primarily for processing and classifying image data.

A basic CNN consists of the following components:

- Convolution Layer
- Activation Layer
- Pooling Layer
- Fully Connected Layer

The convolution operation is represented as:

$$Y(i,j) = \sum_m \sum_n X(i+m, j+n) K(m,n) \tag{1}$$

where:

- X represents the input image.
- K represents the kernel or filter.
- Y represents the resulting feature map.

The output feature-map size is calculated using:

$$O = \left\lfloor \frac{N - F + 2P}{S} \right\rfloor + 1 \tag{2}$$

where:

- N = Input size
- F = Filter size
- P = Padding
- S = Stride

3.1 Common Convolution Kernels

A kernel, also called a filter, is a small matrix that slides over an input image to extract visual features such as edges, textures, corners, and smooth regions.

Table 1: Summary of common convolution kernels

Kernel	**Size**	**Purpose**
Identity	3×3	Preserves the original image
Sobel-X	3×3	Detects vertical edges
Sobel-Y	3×3	Detects horizontal edges
Laplacian	3×3	Detects edges in all directions
Sharpen	3×3	Enhances image details
Box Blur	3×3	Smooths the image and reduces noise
Gaussian Blur	3×3	Reduces noise while preserving structure
Emboss	3×3	Creates a three-dimensional effect
Outline	3×3	Highlights object boundaries
Motion Blur	5×5	Simulates directional motion blur

3.1.1 Identity Kernel

The identity kernel preserves the original image without modifying its pixel values.

$$K = \begin{bmatrix} 0 & 0 & 0 \\ 0 & 1 & 0 \\ 0 & 0 & 0 \end{bmatrix}$$

3.1.2 Sobel-X Kernel

The Sobel-X kernel detects vertical edges by measuring intensity changes along the horizontal direction.

$$K = \begin{bmatrix} -1 & 0 & 1 \\ -2 & 0 & 2 \\ -1 & 0 & 1 \end{bmatrix}$$

3.1.3 Sobel-Y Kernel

The Sobel-Y kernel detects horizontal edges.

$$K = \begin{bmatrix} -1 & -2 & -1 \\ 0 & 0 & 0 \\ 1 & 2 & 1 \end{bmatrix}$$

3.1.4 Laplacian Kernel

The Laplacian kernel detects edges in multiple directions.

$$K = \begin{bmatrix} 0 & 1 & 0 \\ 1 & -4 & 1 \\ 0 & 1 & 0 \end{bmatrix}$$

3.1.5 Sharpen Kernel

The sharpening kernel enhances fine details in an image.

$$K = \begin{bmatrix} 0 & -1 & 0 \\ -1 & 5 & -1 \\ 0 & -1 & 0 \end{bmatrix}$$

3.1.6 Box Blur Kernel

The Box Blur kernel smooths an image by averaging neighboring pixels.

$$K = \frac{1}{9}\begin{bmatrix} 1 & 1 & 1 \\ 1 & 1 & 1 \\ 1 & 1 & 1 \end{bmatrix}$$

3.1.7 Gaussian Blur Kernel

Gaussian blur smooths the image while preserving the overall structure better than a simple average filter.

$$K = \frac{1}{16}\begin{bmatrix} 1 & 2 & 1 \\ 2 & 4 & 2 \\ 1 & 2 & 1 \end{bmatrix}$$

3.1.8 Emboss Kernel

The Emboss kernel creates a three-dimensional appearance by emphasizing directional changes in intensity.

$$K = \begin{bmatrix} -2 & -1 & 0 \\ -1 & 1 & 1 \\ 0 & 1 & 2 \end{bmatrix}$$

3.1.9 Outline Detection Kernel

The outline detection kernel highlights object boundaries.

$$K = \begin{bmatrix} -1 & -1 & -1 \\ -1 & 8 & -1 \\ -1 & -1 & -1 \end{bmatrix}$$

3.1.10 Motion Blur Kernel

The motion blur kernel simulates motion blur along a diagonal direction.

$$K = \frac{1}{5}\begin{bmatrix} 1 & 0 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 & 0 \\ 0 & 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 0 & 1 \end{bmatrix}$$

4 Numerical Examples

4.1 Numerical Example 1: Convolution

Consider the following input matrix:

$$X = \begin{bmatrix} 1 & 2 & 3 \\ 4 & 5 & 6 \\ 7 & 8 & 9 \end{bmatrix}$$

and the kernel:

$$K = \begin{bmatrix} 1 & 0 \\ 0 & 1 \end{bmatrix}$$

The first position is:

$$1(1) + 2(0) + 4(0) + 5(1) = 6$$

The second position is:

$$2(1) + 3(0) + 5(0) + 6(1) = 8$$

The third position is:

$$4(1) + 5(0) + 7(0) + 8(1) = 12$$

The fourth position is:

$$5(1) + 6(0) + 8(0) + 9(1) = 14$$

Therefore, the resulting feature map is:

$$\boxed{\begin{bmatrix} 6 & 8 \\ 12 & 14 \end{bmatrix}}$$

4.2 Numerical Example 2: Max Pooling

Consider the feature map:

$$\begin{bmatrix} 1 & 5 & 2 & 3 \\ 7 & 8 & 1 & 0 \\ 4 & 6 & 9 & 5 \\ 2 & 3 & 1 & 8 \end{bmatrix}$$

Using a 2×2 max-pooling filter:

$$\begin{bmatrix} 1 & 5 \\ 7 & 8 \end{bmatrix} \rightarrow 8$$

$$\begin{bmatrix} 2 & 3 \\ 1 & 0 \end{bmatrix} \rightarrow 3$$

$$\begin{bmatrix} 4 & 6 \\ 2 & 3 \end{bmatrix} \rightarrow 6$$

$$\begin{bmatrix} 9 & 5 \\ 1 & 8 \end{bmatrix} \rightarrow 9$$

Therefore, the output is:

$$\boxed{\begin{bmatrix} 8 & 3 \\ 6 & 9 \end{bmatrix}}$$

4.3 Numerical Example 3: Parameter Calculation

Suppose the input image has dimensions:

$$32 \times 32 \times 3$$

and the convolution layer contains 16 filters with kernel size 3×3.
The number of trainable parameters is:

$$\begin{aligned} \text{Parameters} &= (3 \times 3 \times 3 + 1) \times 16 && (3) \\ &= (27 + 1) \times 16 && (4) \\ &= \boxed{448} && (5) \end{aligned}$$

5 Dataset

The dataset used for this experiment is the CIFAR-10 dataset.
The dataset consists of:

- Training Images: 50,000
- Testing Images: 10,000
- Number of Classes: 10
- Image Size: $32 \times 32 \times 3$

The ten classes are:

Airplane, Automobile, Bird, Cat, Deer, Dog, Frog, Horse, Ship, Truck.

6 Experimental Procedure

6.1 Task 1: CIFAR-10 Dataset Visualization

The CIFAR-10 dataset was loaded using TensorFlow/Keras. Ten sample images were displayed and the class distribution was plotted.

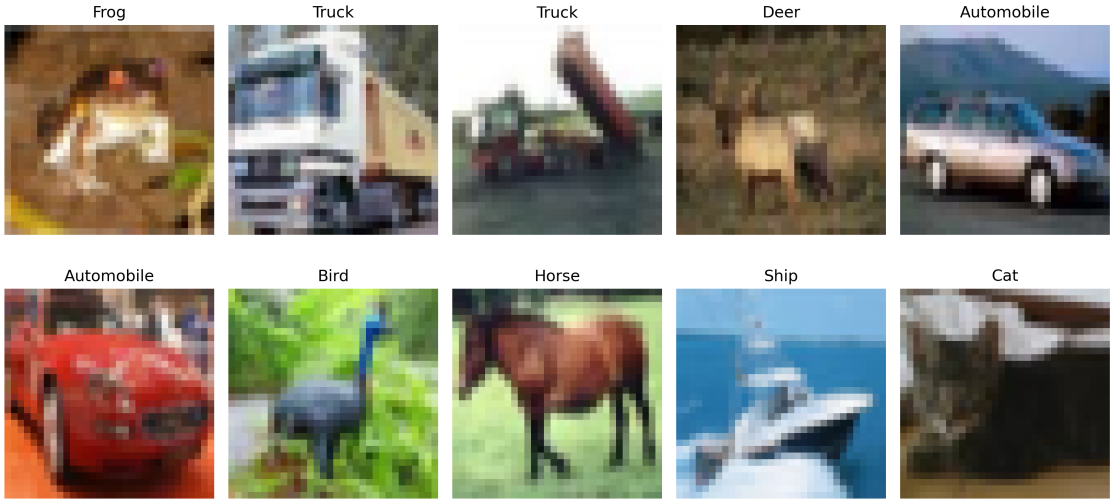

Figure 1: Sample images from the CIFAR-10 dataset

Inference:

The sample images demonstrate different object categories present in the CIFAR-10 dataset. Each image is a 32×32 RGB image belonging to one of ten classes.

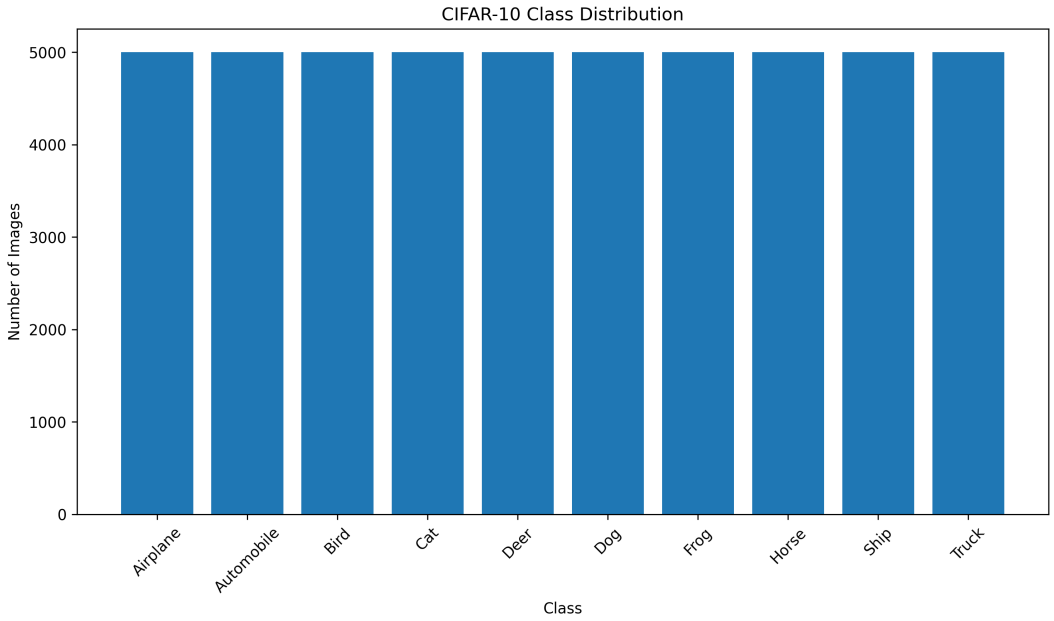

Figure 2: CIFAR-10 class distribution

Inference:

The class distribution is approximately balanced, with around 5,000 training images for each class. Therefore, the dataset does not have a major class imbalance.

6.2 Task 2: Convolution Kernel Comparison

Three convolution kernel sizes were compared:

- 3×3
- 5×5
- 7×7

Valid padding and stride 1 were used.

Table 2: Feature-map dimensions for different kernel sizes

Kernel Size	Input Size	Output Size
3×3	32×32	30×30
5×5	32×32	28×28
7×7	32×32	26×26

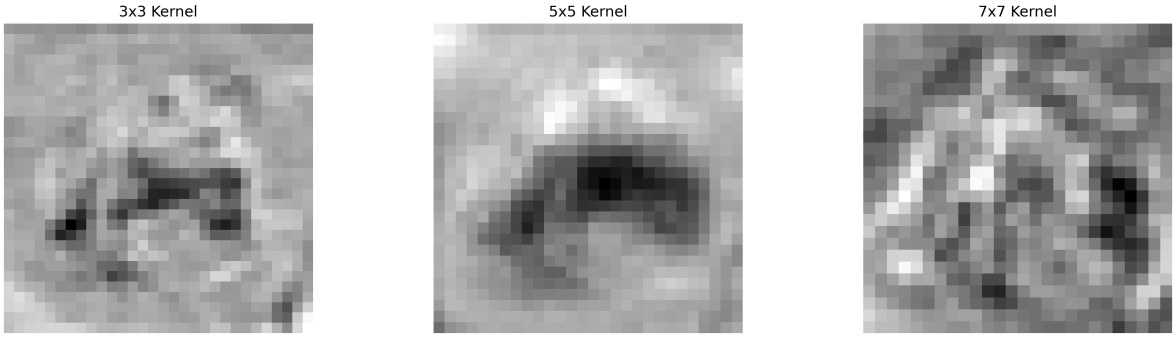

Figure 3: Comparison of convolution feature maps

Inference:
Increasing the kernel size reduces the spatial dimensions of the feature map when valid padding is used. Larger kernels cover a larger receptive field but produce smaller output feature maps.

6.3 Task 3: Stride and Padding

The effects of stride and padding were studied using a 3×3 kernel.

Table 3: Effect of stride and padding

Padding	Stride	Output Size
Valid	1	30×30
Valid	2	15×15
Same	1	32×32

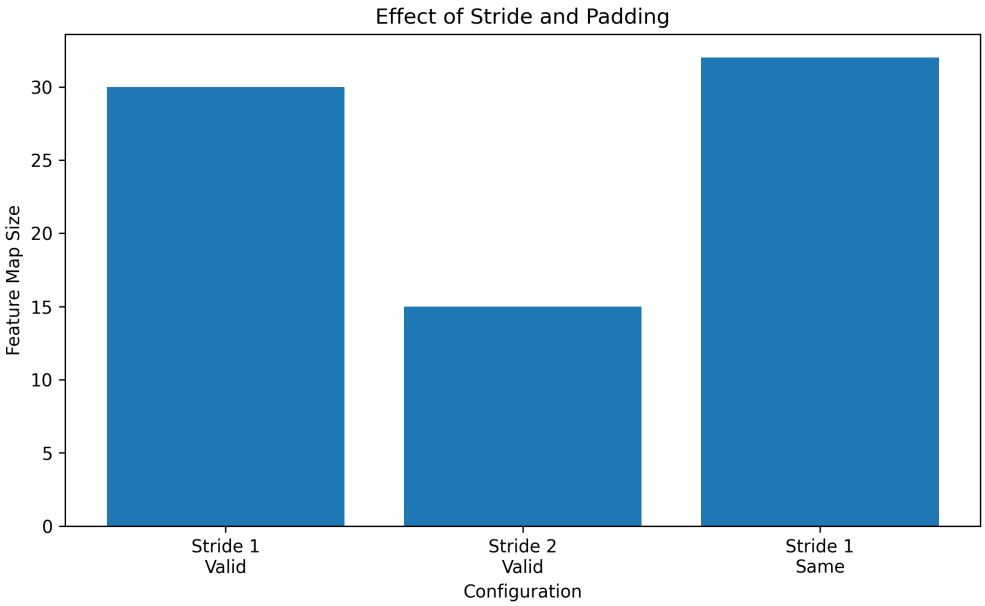

Figure 4: Effect of stride and padding

Inference:

Increasing the stride decreases the spatial dimensions of the output feature map. Same padding preserves the spatial dimensions for stride 1, whereas valid padding reduces the output dimensions.

6.4 Task 4: Feature Map Visualization

A convolution layer containing 16 filters with kernel size 3×3 was applied to an input image.

The resulting feature tensor had dimensions:

$$1 \times 32 \times 32 \times 16$$

At least eight feature maps were visualized.

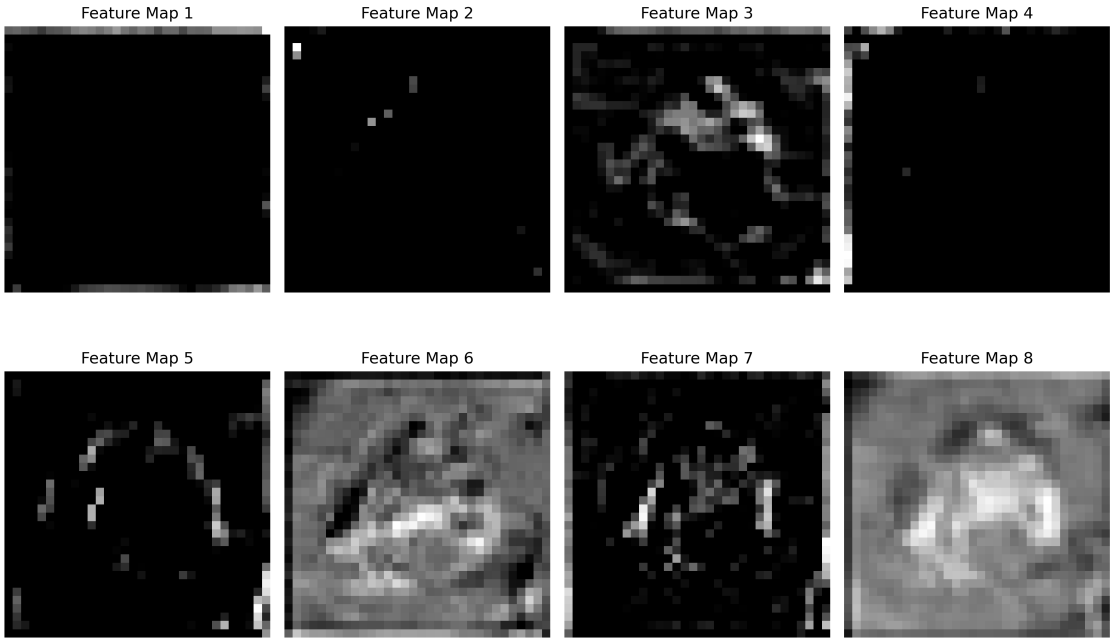

Figure 5: Feature maps generated by the first convolution layer

Inference:

Each feature map represents the response of a convolution filter to different visual patterns in the input image. Different filters can respond to edges, textures, and other local characteristics.

6.5 Task 5: Max Pooling and Average Pooling

A 2×2 pooling window with stride 2 was used to compare max pooling and average pooling.

Table 4: Pooling output dimensions

Operation	**Output Shape**
Original Feature Map	$1 \times 32 \times 32 \times 16$
Max Pooling	$1 \times 16 \times 16 \times 16$
Average Pooling	$1 \times 16 \times 16 \times 16$

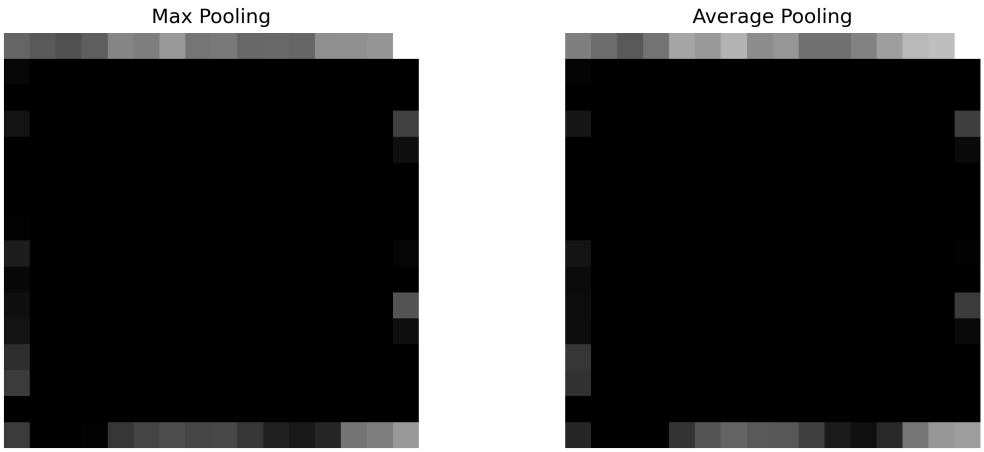

Figure 6: Comparison of max pooling and average pooling

Inference:

Both pooling operations reduce the spatial dimensions by a factor of two. Max pooling retains the strongest activation in each window, whereas average pooling calculates the mean activation.

6.6 Task 6: CNN Construction and Training

The CNN architecture used in this experiment is:

Input → Conv → ReLU → MaxPool → Conv → ReLU → MaxPool → Flatten → Dense → Softmax

The model was trained using:

- Optimizer: Adam
- Epochs: 20
- Batch Size: 32
- Loss Function: Sparse Categorical Crossentropy

The final model contained:

$$\boxed{545098}$$

trainable parameters.

6.7 Task 7: Model Evaluation

The trained CNN was evaluated using:

- Accuracy
- Precision
- Recall

- F1-score
- Confusion Matrix
- Classification Report

7 Mandatory Plots

7.1 Training Accuracy

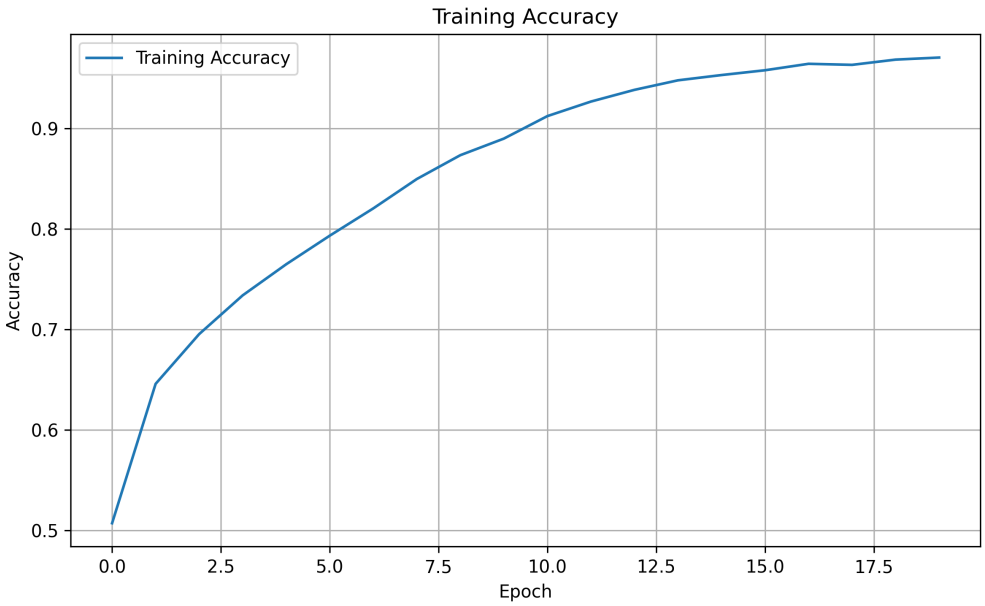

Figure 7: Training accuracy

Inference:

The training accuracy increased significantly over the training epochs and reached approximately 97.06%. This indicates that the model learned the training dataset effectively.

7.2 Validation Accuracy

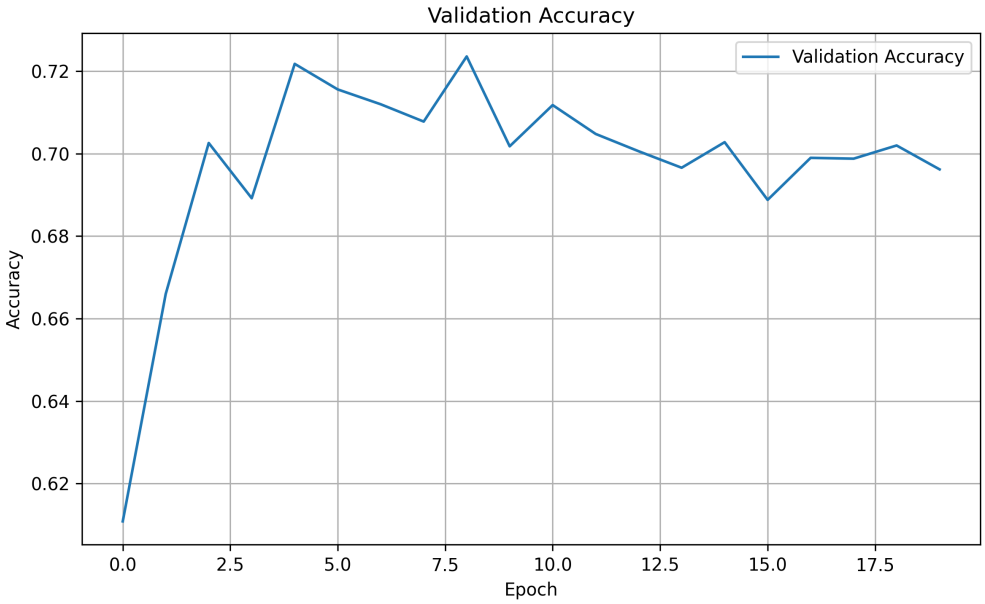

Figure 8: Validation accuracy

Inference:

The validation accuracy reached approximately 69.62%. The difference between training and validation accuracy indicates that the model has some degree of overfitting.

7.3 Training Loss

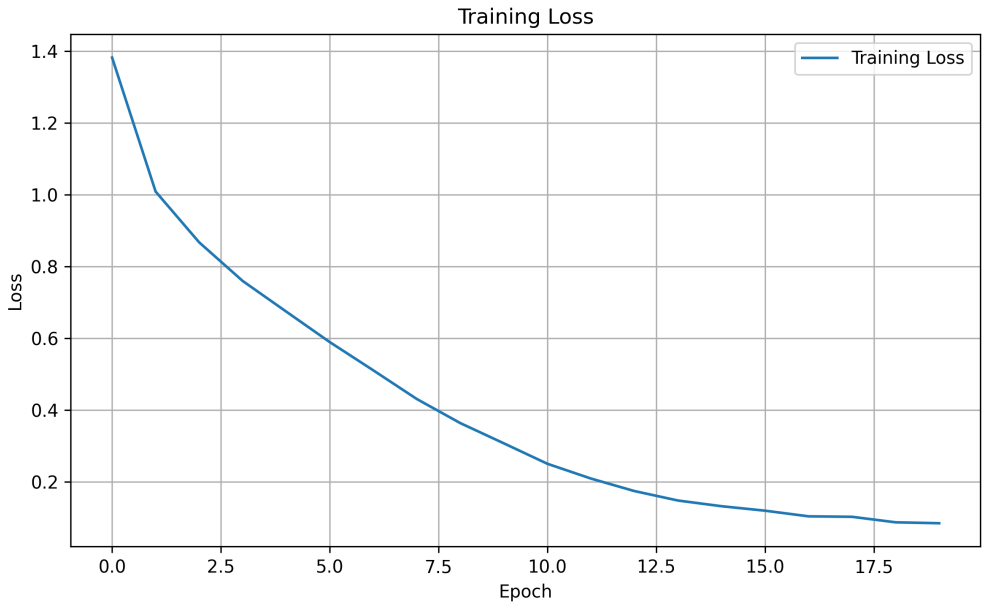

Figure 9: Training loss

Inference:

The training loss decreases as the model learns the patterns in the training data. This indicates that the optimization process is successfully reducing the classification error on the training set.

7.4 Validation Loss

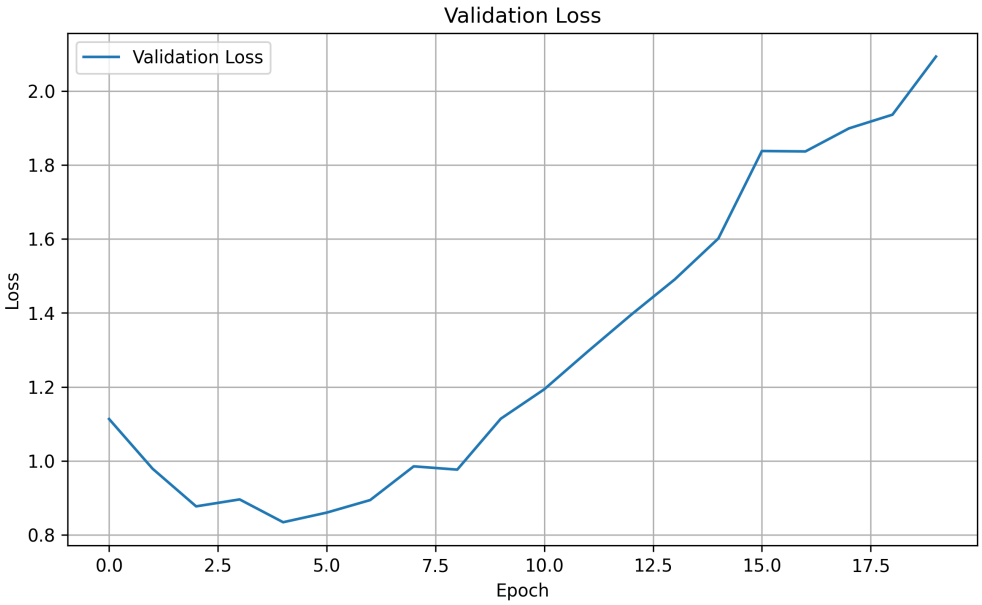

Figure 10: Validation loss

Inference:

The validation loss does not follow the same decreasing trend as training loss toward the end of training. This suggests that the model begins to overfit the training data.

7.5 Feature Maps

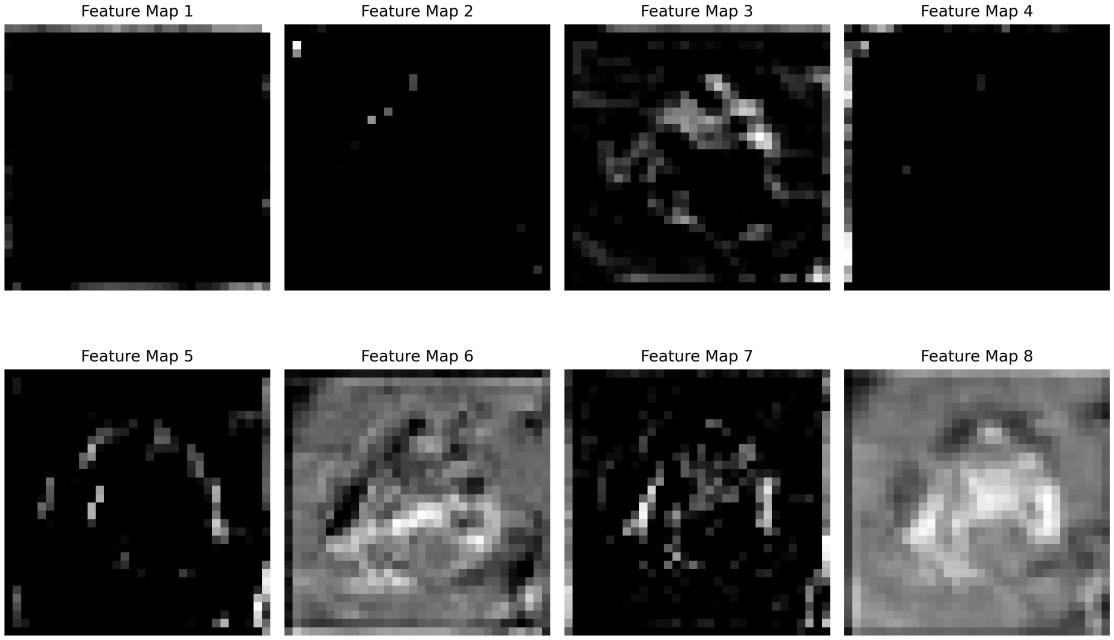

Figure 11: Feature maps from the first convolution layer

Inference:

The feature maps demonstrate how different convolution filters respond to visual patterns in the input image. The filters produce different activation patterns based on the features they detect.

7.6 Confusion Matrix

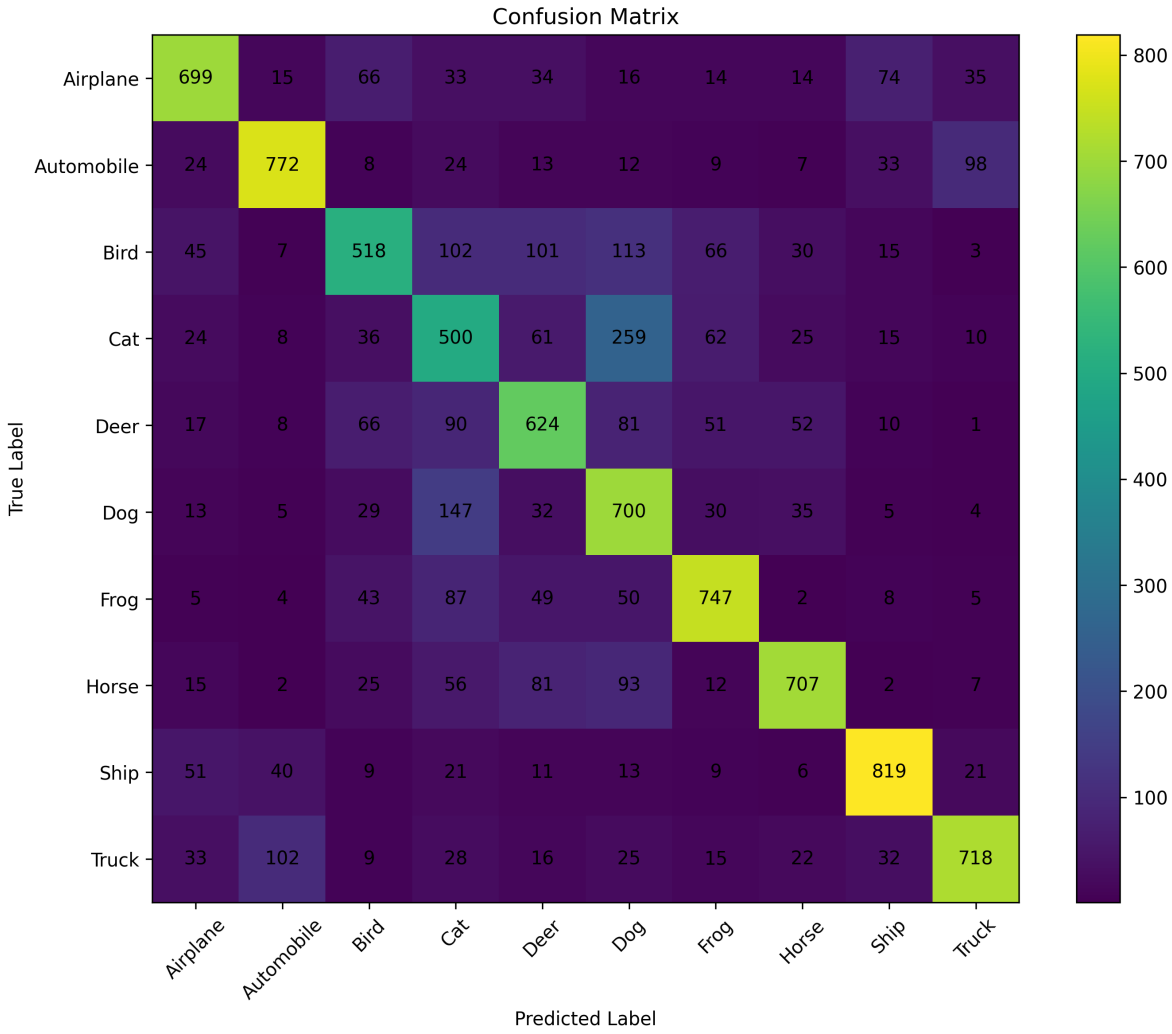

Figure 12: Confusion matrix for CIFAR-10 classification

Inference:

The confusion matrix shows the classification performance for each of the ten CIFAR-10 classes. Higher values along the diagonal represent correctly classified samples, while off-diagonal values represent misclassifications.

7.7 Convolution Kernel Comparison

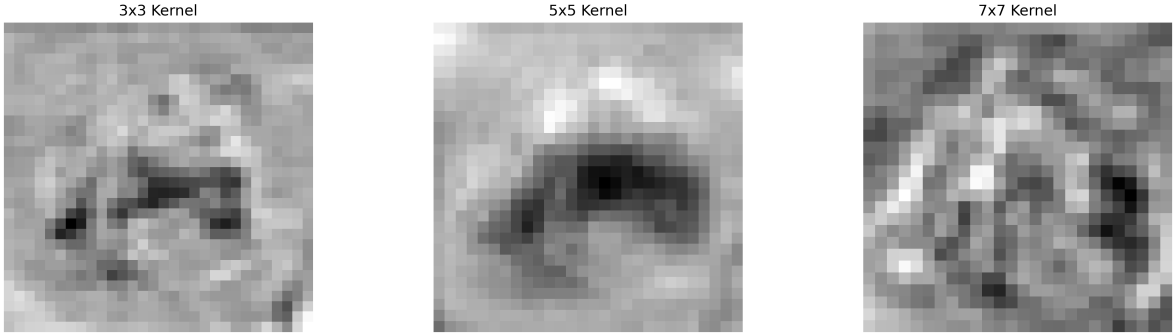

Figure 13: Comparison of convolution outputs for different kernel sizes

Inference:

The convolution outputs change with the kernel size. Larger kernels capture information from a wider local region but also reduce the spatial dimensions when valid padding is used.

7.8 Stride and Padding Comparison

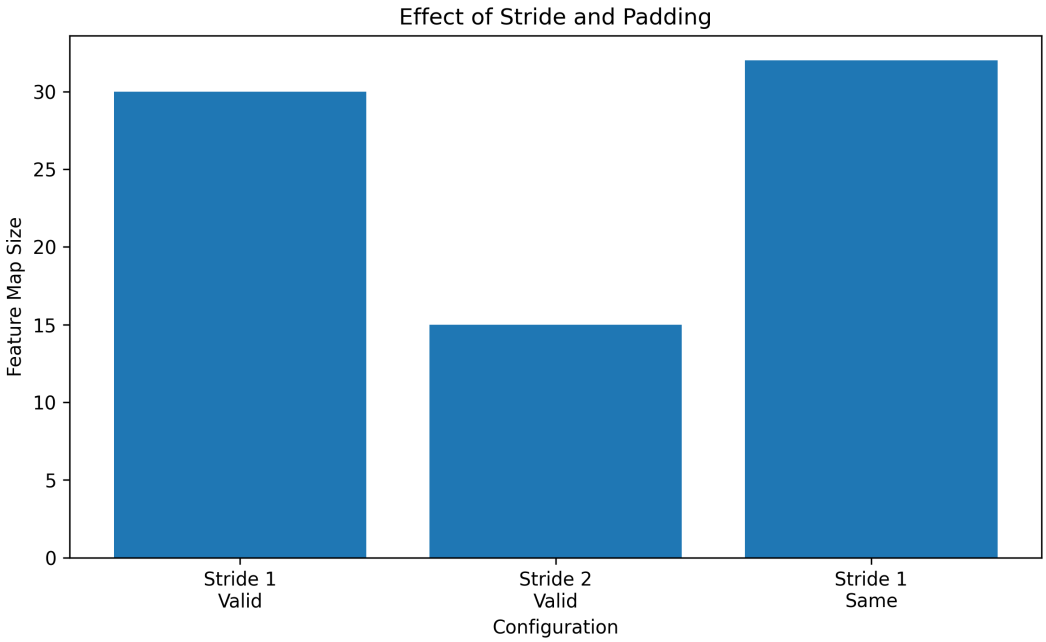

Figure 14: Effect of stride and padding on feature-map dimensions

Inference:

Increasing the stride reduces the spatial dimensions of the output. Same padding helps preserve the spatial dimensions, whereas valid padding produces a smaller feature map.

8 Results

The final CNN performance is summarized below.

Table 5: Final CNN performance

Metric	Value
Training Accuracy	97.06%
Validation Accuracy	69.62%
Testing Accuracy	68.04%
Precision	69.07%
Recall	68.04%
F1-score	68.28%
Test Loss	2.2466
Trainable Parameters	545,098

8.1 Classification Report

Table 6: Class-wise classification performance

Class	Precision	Recall	F1-score
Airplane	0.75	0.70	0.73
Automobile	0.80	0.77	0.79
Bird	0.64	0.52	0.57
Cat	0.46	0.50	0.48
Deer	0.61	0.62	0.62
Dog	0.51	0.70	0.59
Frog	0.74	0.75	0.74
Horse	0.79	0.71	0.74
Ship	0.81	0.82	0.81
Truck	0.80	0.72	0.75

9 Additional Exercises

9.1 1. Output Size Calculation

Calculate the output size for a 64×64 image using a 5×5 kernel with stride 2 and padding 2.

Using:

$$O = \left\lfloor \frac{N - F + 2P}{S} \right\rfloor + 1$$

we get:

$$O = \left\lfloor \frac{64 - 5 + 2(2)}{2} \right\rfloor + 1 \tag{6}$$

$$= \left\lfloor \frac{63}{2} \right\rfloor + 1 \tag{7}$$

$$= 31 + 1 \tag{8}$$

$$= \boxed{32} \tag{9}$$

Therefore, the output size is:

$$\boxed{32 \times 32}$$

9.2 2. Trainable Parameters

For 64 filters of size 3×3 and an RGB input:

$$P = (3 \times 3 \times 3 + 1) \times 64 \tag{10}$$

$$= 28 \times 64 \tag{11}$$

$$= \boxed{1792} \tag{12}$$

Therefore, the convolution layer contains 1,792 trainable parameters.

9.3 3. ReLU and Sigmoid Activation Functions

The ReLU activation function is:

$$f(x) = \max(0, x)$$

ReLU returns zero for negative values and the input value for positive values. It is commonly used in hidden layers of CNNs.

The sigmoid activation function is:

$$\sigma(x) = \frac{1}{1 + e^{-x}}$$

Sigmoid maps values into the range $(0, 1)$. It can be useful for probability outputs, but it may suffer from vanishing gradients.

9.4 4. Max Pooling and Average Pooling

Max pooling selects the maximum activation from each pooling window. Average pooling calculates the average value of the activation values in the window.

Max pooling generally preserves the strongest detected feature, while average pooling produces a smoother representation.

In this experiment, both operations reduce a 32×32 feature map to 16×16 when a 2×2 window and stride 2 are used.

9.5 5. Increasing Filters from 16 to 64

Increasing the number of convolution filters from 16 to 64 allows the CNN to learn a larger number of feature representations.

However, increasing the number of filters also increases:

- Number of trainable parameters
- Memory usage
- Computational cost
- Training time

The effect on classification accuracy and computation time should be determined experimentally by retraining the model.

10 Discussion

10.1 1. Why is convolution preferred over fully connected layers for images?

Convolution is preferred because it exploits the spatial structure of images through local receptive fields and shared weights.

Instead of connecting every pixel to every neuron, convolution filters operate over small local regions. This significantly reduces the number of parameters and allows the network to learn spatial features efficiently.

10.2 2. How does stride affect the feature map size?

Stride determines how far the convolution kernel moves across the input image at each step.

A larger stride causes the kernel to skip more positions and therefore produces a smaller feature map.

10.3 3. What is the role of padding?

Padding adds additional pixels around the boundary of the input image.

Same padding can preserve the spatial dimensions of the input when stride is 1, while valid padding does not add extra pixels and generally produces a smaller output feature map.

10.4 4. Why is pooling used?

Pooling reduces the spatial dimensions of feature maps and therefore reduces computational requirements.

It also helps retain important information while providing a degree of spatial invariance.

10.5 5. How do feature maps represent image characteristics?

Feature maps represent the responses produced by convolution filters.

Different filters can respond to different visual patterns such as edges, textures, shapes, and other local image characteristics.

10.6 6. Why do CNNs require fewer parameters than MLPs?

CNNs use local connectivity and weight sharing.

The same convolution filter is applied at different spatial locations, meaning that the network does not require separate weights for every pixel-to-neuron connection.

Therefore, CNNs generally require significantly fewer parameters than fully connected MLPs for image processing.

11 Conclusion

A Convolutional Neural Network was successfully implemented for CIFAR-10 image classification using TensorFlow/Keras.

The experiment demonstrated the convolution operation, comparison of kernel sizes, stride and padding, feature map visualization, max pooling, average pooling, CNN model training, and model evaluation.

The final model achieved a testing accuracy of:

$$\boxed{68.04\%}$$

with the following performance:

- Training Accuracy: 97.06%
- Validation Accuracy: 69.62%
- Testing Accuracy: 68.04%
- Precision: 69.07%
- Recall: 68.04%
- F1-score: 68.28%
- Trainable Parameters: 545,098

The difference between training and testing accuracy indicates that the model has a noticeable generalization gap and could potentially benefit from techniques such as data augmentation, regularization, dropout, or additional tuning.

Source Code

The complete implementation of the Convolutional Neural Network (CNN) for CIFAR-10 image classification, including dataset visualization, convolution operations, stride and padding analysis, feature map visualization, max pooling, average pooling, CNN model training, evaluation, and visualization, has been uploaded to GitHub.

The source code can be accessed at:

`https://github.com/MouryaAditya999/Deep-Learning-Lab/tree/main/Experiment_3`

The repository contains:

- Jupyter Notebook containing the complete CNN implementation
- CIFAR-10 dataset loading and preprocessing code
- Convolution, stride, and padding experiments
- Feature map and pooling visualizations
- CNN model training and evaluation
- Generated plots and experimental results
- Experiment report
- README file with execution instructions
- `requirements.txt` containing the required Python packages

12 References

1. Goodfellow, I., Bengio, Y., and Courville, A., *Deep Learning*.
2. Bishop, C. M., *Pattern Recognition and Machine Learning*.
3. Haykin, S., *Neural Networks and Learning Machines*.
4. TensorFlow Documentation.
5. CIFAR-10 Dataset Documentation.